# CS251: Introduction to Language Processing

**End-semester Examination (2023-24 M Semester)**

**Max. Points:** 100 **Duration:** 2 hours

December 8, 2023

**Instructions**

- Write down your assumptions wherever necessary.
- All the questions are compulsory.

## Question-1 (Semantic Analysis)

[**12 Points**]

For any given valid arithmetic expression containing the operators *, +, -, write syntax directed definition (SDD) that prints the postfix representation of the expression.

**Example:** For the given arithmetic expression $(5 + 3) * (8 - 2)$ your SDD should print $53 + 82 - *$. Assume that the operator precedence and associativity follow that of the C language.

## Question-2 (Three Address Code Generation)

[**25 Points**]

Assume that your programming language supports control flow instructions for the "For Loops" using the following grammar

$S -> for\ (E; E; E)\ S$

**Part-(a):** Write down the semantic rules to generate the three address code as intermediate representation.
*Hints:* (1) You can use the backpatch function discussed in the class. (2) You can introduce the markers (M, N, etc) into the above grammar with suitable epsilon transitions to pick up index of next quadruple

**Part-(b):** Using the above semantic rules, generate three address code for the following snippet of the code.

```
for ( i=0;  i<x+y ;  i=i+1)  {
        i=i*x+y ;
}
```

# Question-3 (Code Generation)

**[23 Points]**

Assume that we have the following target machine model.

| **Instruction** | **Semantics** |
|---|---|
| Store $m$, $r$ | Stores the contents of register $r$ to the memory location $m$ |
| Load $r$, $m$ | Loads the contents of the memory location $m$ to the register $r$ |
| OP $r2$, $r2$, $r1$ | Performs the binary operation of two register operands $r2$ and $r1$.<br>The result of $r2$ OP $r1$ is stored in $r2$. |

Note that above model requires all the operands of binary operation to be registers.

**(a)** Modify the labeling algorithm that we discussed in the class to calculate the minimum number of registers required to compute an expression tree supporting the above target machine model.

**(b)** Use the modified labeled algorithm and compute the minimum number of registers required to compute the following expression.

**a*b+c/(d*e)**

You must draw the expression tree and list the label for each node in the tree. Otherwise, you will not get any points.

**(c)** Assume that the target machine has only two registers. Write the final sequence of assembly instructions that are generated for the expression in Part-(b) on the target machine model.

# Question-4 (Bottom-up Parsing (LR1))

**[25 Points]**

Consider the following grammar:

$$
\begin{array}{lcl}
S & : & E \\
E & : & T - E \\
 & | & T \\
T & : & --T \\
 & | & int
\end{array}
$$

Answer the following:

1. Construct the DFA for LR(1)
2. Construct parser table for LR(1)
3. Is the above grammar LR(1)? Justify your answer.

# Question-5 (Machine Independent Optimizations)

**[15 Points]**

Consider the following three address code.

```
t1=j-1
t2=4*t1
t3=A[t2]
t4=j+1
t5=t4-1
t6=4*t5
t7=A[t6]
t8=j-1
t9=4*t8
A[t9]=t7
t10=j+1
t11=t10-1
t12=4*t11
A[t12]= t3
```

Perform the following optimizations any number of times to generate the final optimized three-address code. Note that you must apply one optimization at a time and show the transformed three-address code after each step.

1. Global common sub-expression elimination
2. Copy propagation
3. Dead code elimination – which removes the useless statements from the three-address code.